

HeroQuest

Author: Đoàn Minh Đức

Contents

- **Overview:**

- Game Idea:

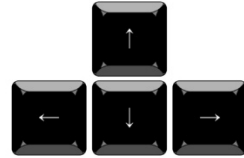
- **HeroQuest** is a 2D platformer game where the player controls the main character (the Hero) to jump, fight, and shoot to progress through rounds, collect keys or coins, avoid obstacles, and defeat enemies. The game features multiple rounds, each with increasing difficulty.



- **Game Design:**

- **Input Controls:**

- Arrow keys vector



- Space



- Mouse



- Escape key

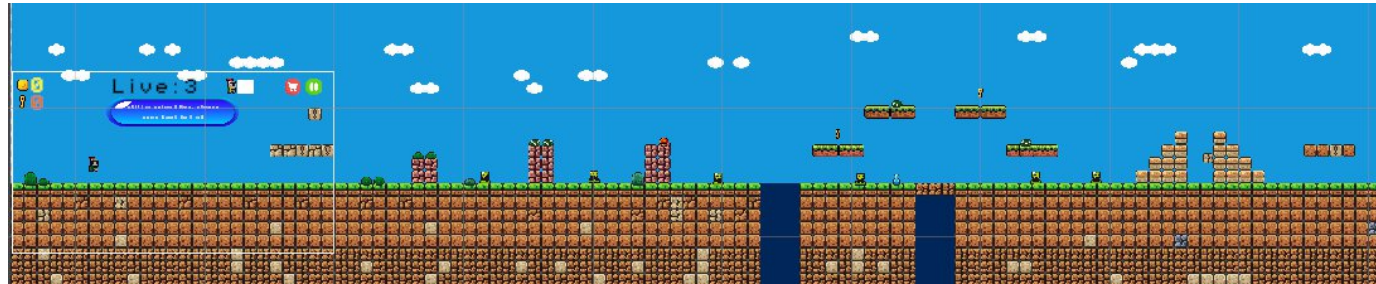


KeyBoard input	Action
Up arrow	Jump
Down arrow	Sit
Left arrow	Move left
Right arrow	Move right
Space	shoot or slash
Mouse	Click UI
Escape key	Pause game

- **Game Play:**

- **Game-world Layout:**

- Platforms (Ground/Blocks), Gates, Obstacles, Traps, and Enemy Spawn Points.
 - The Hero starts at the Start point, collects keys to open Gates, and progresses to the next round.
 - Safe zones allow the Hero to recover or plan movements, while dangerous zones and traps challenge the player's skills.
 - Checkpoints may be placed in longer rounds to save progress.
 - Power-ups and items are scattered throughout the map to aid the Hero in completing rounds.



- **Menu Layout:**

- **Main Menu** includes the following main options:

- Start: Starts the game and takes the player to the round selection screen.
 - Setting: Opens the settings screen, allowing the player to adjust the volume.
 - Exit: Quits the game.
 - The game title (**HeroQuest**) is placed at the top center, ensuring easy brand recognition.
 - Menu buttons are arranged vertically in a straight line, with large sizes to ensure easy clicking on both PC and touch screens.
 - Optional hover/click effects can be added to enhance player interaction experience.

- **Level Selection Menu Layout:**

- Function: Allows the player to select the desired level to start.
- Layout:
 - Level icons are arranged in a 3×3 grid, making it easy for players to view and quickly choose a level.
 - Each level icon can display the level number and status:
 - Unlocked: Bright colors, selectable.
 - Locked: Dimmed/gray with a lock icon, not selectable.
 - Back button (top left corner) to return to the Main Menu.
 - Left/Right navigation buttons (red) to switch between level pages when the total number of levels exceeds the current page.

- **Settings Menu Layout:**

- Function: Allows the player to adjust the game's audio settings.
- Layout:
 - Music Slider: Adjusts the background music volume. A red music note icon on the left helps players quickly recognize its function.
 - Effect Slider: Adjusts the sound effects (SFX) volume. A yellow speaker icon is placed on the left.
 - Close Button: A white "X" inside a green circle at the top right corner, used to return to the previous screen.

- **Pause Menu Layout:**

- Function: Allows the player to pause the game and choose the next action.
- Layout:
 - Title “Pause” at the top, using a large, clear pixel font.
 - A horizontal row of function buttons including:
 - Home (red house icon): Return to the Main Menu.
 - Resume (blue play icon): Continue the game.
 - Restart (green circular arrow icon): Restart the current level from the beginning.
 - Settings (yellow music note icon): Open the Settings Menu to adjust music and sound effects.
 - Close button at the top right corner to exit the pause menu and return to gameplay.

- **Shop Menu Layout:**

- Function: Allows the player to purchase items using in-game currency (coins).
- Layout:
 - Top-right corner: A green Close button to return to the game.
 - Item display area:
 - Each item is presented in its own individual slot, including:
 - Item icon (e.g., shooting skill level 1, sword skill level 2, fruit level 1 and level 2).

- **Win Screen Layout:**

- Function: Displays when the player successfully completes a round.
- Layout:
 - Title: "You Win!!" in bold black font, clearly announcing victory.
 - Function buttons:
 - Next Round: Proceeds to the next round.
 - Main Menu: Returns to the main menu.

- **Game Over Screen Layout:**

- Function: Displays when the player loses (runs out of health or time).
- Layout:
 - Title: "Game Over" in black font, positioned at the top center.
 - Function buttons:
 - Play Again: Restarts the current round from the beginning.
 - Main Menu: Returns to the main menu.

- **Architecture:**

- **Overall Structure**

- Engine: Unity.
 - Design Pattern:
 - Prefab-based workflow: All objects (player, enemies, items, UI elements) are packaged as Prefabs for easy reuse and updates.
 - LeanTween: Used for UI tweening, transition effects, and lightweight animations without complex Animators.
 - Asset Organization: Assets (sprites, audio, tilemaps) are organized into dedicated folders (e.g., Assets/Sprites, Assets/UI, Assets/Audio).

- Core Systems
 - Character System
 - PlayerController script handles movement, jumping, and collision detection.
 - Uses Rigidbody2D and Collider2D for physics interactions.
 - Animations managed through an Animator Controller, triggered via scripts.
 - Enemies are Prefabs with simple (attacking).
 - UI System
 - HUD: Displays lives, coins, and health bar using Unity UI.
 - Menus & Level Selection: Built with Canvas and ScrollView, enhanced with LeanTween for smooth transitions.
 - Pause/Shop Menus: Toggled using keyboard or click.
 - Audio System
 - AudioManager: Controls background music (BGM) and sound effects (SFX).
 - Level Management
 - Each level is a separate Scene containing Tilemaps and Spawn Points.
 - Loading Scene provides smooth level transitions.

- Cutsценe / Timeline System
 - Unity Timeline is used for cutscenes.
 - Characters and cameras in cutscenes are controlled by PlayableDirector.

- **Implementation**

- Engine & Build: Unity 2D for PC.
- Project Structure: Prefab-based workflow, LeanTween for animations, and Scene management.
- Programming Approach: C# language, organized code structure, and Git workflow (if applicable).
- Development Timeline: Feature implementation order (e.g., Player → UI → Audio → Levels → Cutscenes).
- Roles / Assignments
 - Game Designer – Creates gameplay concepts and level designs.
 - Programmer – Codes the game in Unity using C#.
 - Artist – Creates sprites, tilemaps, and UI elements.
 - Audio Designer – Produces sound effects and background music.
- Assets / Tools / Tutorials
 - Assets: Sprites, tilemaps, audio, fonts, etc. (sources: self-created, Unity Asset Store, Freepik, etc.).
 - Tools: Unity Editor, LeanTween, Timeline, Animator, Tilemap, etc.
 - Tutorials: YouTube, GreenAcademy, Google, ChatGPT, etc.

- Techniques
 - Rigidbody2D + Collider2D – For physics handling.
 - Tilemap – Used to create level terrain.
 - Animator + LeanTween – For animations and smooth UI transitions.
 - SceneManager.LoadSceneAsync – For seamless scene transitions.
 - Timeline – Used for cutscenes.
 - AudioManager – To control background music and sound effect volumes.
- Working History / Problems & Solutions
 - Working History (Development Timeline)
 - Week 1: Designed the player character and movement system.
 - Week 2: Created the HUD and menu UI.
 - Week 3: Added audio and cutscenes.
 - Week 4: Performed optimization and testing.
 - Problems & Solutions
 - Problem: Player occasionally fell through the Tilemap when falling at high speed.
 - Solution: Changed Collision Detection to Continuous.
 - Problem: UI stuttered during tween animations.
 - Solution: Used LeanTween with an easing function for smoother transitions.

- **Contents**

- **Pros**

- Smooth gameplay and responsive controls.
 - Clean and visually appealing UI.
 - Good performance optimization for PC.

- **Cons**

- Inconsistent Pixels Per Unit settings caused minor visual issues.
 - Limited character skills and abilities.

- **Lessons Learned**

- Managing prefabs and assets properly makes the project easier to expand.
 - LeanTween simplifies UI animation creation.
 - Gained experience in creating cutscenes using Timeline.
 - Learned how to set up Animator controllers for player, enemies, and bosses.

- **Future Development**

- Add more levels and boss battles.
- Upgrade the enemy AI system.
- Publish the game on Web or Mobile platforms.
- Integrate multiplayer mode or an online leaderboard

- **Overview**

- Game Name: HeroQuest
- Genre: 2D Platformer
- Platform: PC (current), Web (coming soon)

- **Story**

- A brave hero and a princess were living happily together until a cunning thief appeared and kidnapped the princess. Determined to save her, the hero sets out on a perilous journey.
- Along the way, he faces countless challenges—overcoming dangerous obstacles, battling fierce monsters, and leaping across treacherous chasms.
- In the final showdown, the hero confronts the thief in an epic battle, rescues the princess, and after all the hardships, the two are joyfully reunited.

- **Implementation – Tools**

- **Tools**

- Unity Editor – Main game engine and development environment.
 - LeanTween – For smooth UI animations and transitions.
 - Timeline – To create and control cutscenes.
 - Animator – For character, enemy, and boss animations.
 - Tilemap – To design and build level terrains.
 - AudioManager – To manage background music and sound effects.
 - GitHub– For version control and team collaboration.

- **Implementation – Tutorials**

- Unity Learn – Basic tutorials for Unity.
- LeanTween Documentation – Official documentation for LeanTween to learn UI animations.
- Unity Timeline Tutorials (YouTube) – Video guides on creating and controlling cutscenes using Timeline.
- 2D Platformer Tutorials – Tutorials for building a 2D platformer
- Tilemap Tutorials – Lessons on designing and building 2D maps using Tilemap.
- Animator & Animation Tutorials – Guides on creating animations for characters, enemies, and bosses.
- Audio Mixer Tutorials – Tutorials for managing background music and sound effects in Unity.
- Git Basics – Basic tutorials on Git for version control and project collaboration.

• **Implementation – Techniques**

- Prefab Workflow – Use prefabs for reusable objects (characters, enemies, platforms) to speed up level creation and updates.
- Object Pooling – Recycle frequently used objects (like bullets or enemies) to improve performance and reduce memory usage.
- Physics with Rigidbody2D & Collider2D – Enable realistic movement, jumping, and collision detection for characters and objects.
- Tilemap for Level Design – Quickly build and edit large 2D environments.
- Animator & LeanTween for Animations – Combine Animator for character/enemy actions and LeanTween for smooth UI transitions.
- Async Loading with SceneManager.LoadSceneAsync – Create smooth scene transitions with loading screens.
- Timeline for Cutscenes – Script and control story sequences or boss introductions.
- AudioManager for Sound Management – Adjust music, sound effects, and volume levels dynamically.
- Version Control (Git) – Manage project history and track changes safely.